



OutageIQ

Grid Intelligence Dashboard

Predicting equipment failure before it becomes an outage — with an explanation an engineer can act on.

Team GridSynapse

Dhruv Bhoi · Jiya Sheth · Kush Parekh · Aditi Goraniya

Utilities find out after the lights go out

Problem Statement U1 — Utilities

\$1M+

cost per hour of a
major grid outage

Weeks

of warning already sitting
unread in sensor data

Calendar

not condition — how most
maintenance is still scheduled

Why the warning gets missed

- **Sensor data is reported, not interpreted.** Raw temperature, vibration and partial-discharge numbers arrive as readings — never as a ranked list of what to inspect first.
- **Weather and asset health live in different systems.** The compounding risk of "already-degraded equipment + incoming storm" is rarely caught by either team in time to act.
- **Every asset is treated the same.** A 5-year-old transformer in good health gets the same inspection slot as a 22-year-old one with Poor oil and three prior failures.

One score, one reason, one action — per asset

What OutageIQ does

1

Weighted risk score

Every asset gets a single 0–100 score from sensor condition (60%), live weather (20%) and operational criticality (20%).

2

Live weather, not assumed weather

Real-time temperature, wind and precipitation pulled per region from the Open-Meteo API at scoring time.

3

Explainable by design

Each score comes with a paragraph naming the exact readings that caused it — no black box a technician has to trust blindly.

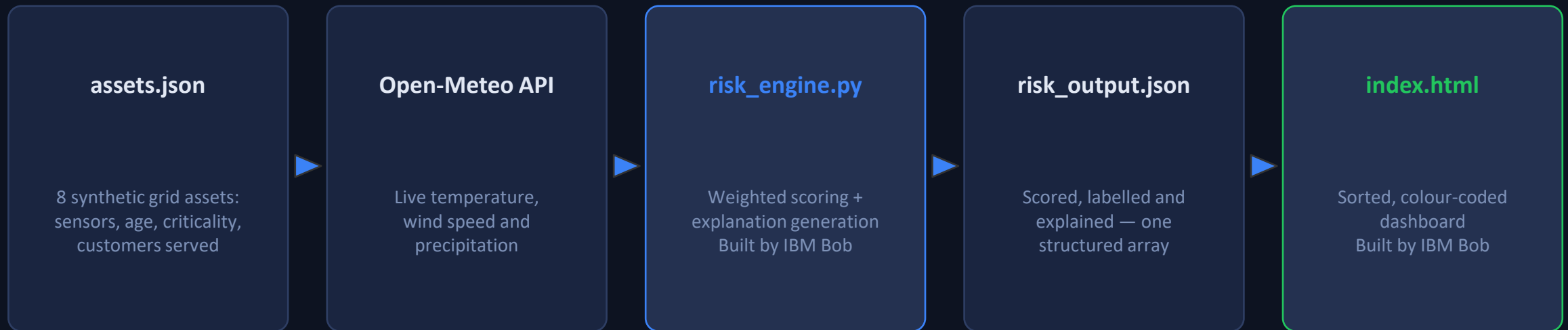
4

A time-bound next step

Critical → inspect within 6 hours. Low → monitor remotely. The output is a work order, not a number.

How it works

End-to-end data flow — no server, no database, no build step

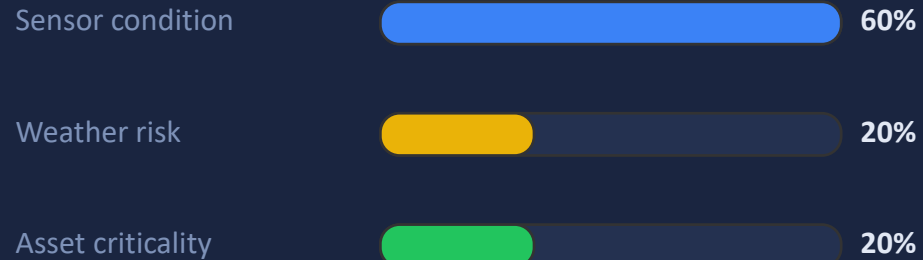


- **Zero dependencies**
One Python file and one HTML file. The only pip install is requests.
- **Runs offline**
The dashboard ships with an embedded data snapshot, so it renders even when opened by double-click.
- **Swappable at the edges**
Replace assets.json with a SCADA feed and none of the scoring logic changes.

The scoring model

Transparent by construction — every score traces back to a reading

Composite weighting



Safe operating thresholds

Temperature	85 °C
Vibration	6 mm/s
Partial discharge	400 pC
Oil quality	Good → Poor scale

Why not just average the sensors?

Averaging lets one healthy reading dilute three dangerous ones. OutageIQ instead takes the worst breach as a floor and stacks the rest on top — so multiple weak signals compound into one strong one, the way real failures actually behave.

```
score = max(sensors)  
       + 0.15 × Σ(rest)
```

What it actually returns

All 8 monitored assets, ranked — real output from risk_output.json



Thresholds: Critical ≥ 80 · High 60–79 · Medium 40–59 · Low < 40

The operator's view

src/index.html — one self-contained page, opens by double-click



Summary bar

Total assets monitored, with live Critical / High / Medium / Low counts, colour-coded.



Risk distribution chart

Animated pure-CSS bar chart showing how the fleet is spread across risk bands.



Ranked asset cards

Sorted by score descending and colour-coded, so the most urgent asset is always first on screen.



Click to expand

Reveals the full explanation under "Why This Matters" plus the recommended action. Keyboard accessible.

Sample output — S-021

81.1 · CRITICAL

Temperature 88 °C exceeds the safe limit of 85 °C, partial discharge 510 pC exceeds the safe limit of 400 pC, and oil quality is Poor. Operational criticality is Critical — it serves 9,800 customers and has 3 prior failures over a 22-year lifespan.

RECOMMENDED ACTION

Inspect within 6 hours and prepare contingency switching plan

Beyond scoring: an AI advisor and a trend view

What's new in this build

1

AI Advisor chat

A chat panel backed by a lightweight Flask API lets an operator ask natural-language questions — grounded in the live risk data via watsonx.ai.

2

Historical trend sparklines

Each asset's expanded card shows a small trend chart of its recent risk scores, so drift toward failure is visible at a glance.

3

Search, filter & sort

Operators can search by asset ID or risk tier, and re-sort the list by score, name, or criticality — no more scanning fixed order.

4

Fails safely, always

If the backend isn't running, the chat shows a clear offline message. Nothing else on the dashboard is affected.

IBM Bob wrote this project

Agent mode in VS Code — the builder, not a mentioned tool

1

Wrote risk_engine.py

From a natural-language spec: weather fetch, weighted scoring, labelling, explanation generation and the console summary table — in one Agent-mode task session.

2

Wrote index.html

The full dashboard in a second session, including the embedded-data fallback so the page still renders when opened directly from the filesystem.

3

Diagnosed its own bug

Asked why a clearly critical substation scored only 48.9, Bob traced it to two causes and rewrote the combination formula itself. Later adapted the watsonx.ai integration to a currently available Granite model after the originally specified one was deprecated.

Tech stack

IBM technology

IBM Bob (Agent mode), watsonx.ai (Granite)

Languages

Python, JavaScript, HTML/CSS

Frameworks

Flask — AI advisor chat backend only

Data

Synthetic JSON dataset

External API

Open-Meteo (free, keyless)

CI

GitHub Actions validation

Every task session is exported to bob_sessions/ with its consumption summary, per the hackathon's compliance requirements.

Decisions we made on purpose

And what each one cost us

1

Rule-based scoring over a trained model

Grid maintenance is safety-critical. A judge, or an engineer, can trace any score back to the exact reading that produced it.

Cost: no learned patterns — the thresholds are ours, not the data's.

2

Framework-free core, one optional backend

The scoring engine and dashboard remain one Python file and one HTML file. A small Flask server was added solely to power the AI advisor chat, kept fully separate so its absence never affects the rest of the dashboard.

Cost: one extra dependency (Flask), scoped to a single optional feature.

3

Synthetic data, honestly labelled

Eight hand-authored assets with a deliberate spread of healthy and failing, so the ranking is demonstrable in 30 seconds.

Cost: the model is unvalidated against real failure history.

What it doesn't do yet

Honest scope, and the path past it

Current limitations

- Scoring is a weighted rule set, not a model trained on failure history.
- The dataset is synthetic — it is not wired to a live sensor feed.
- Runs locally only; no cloud deployment, per hackathon rules.
- Single-operator scope — no authentication, no multi-user state.
- Weather is fetched per region, not per individual asset location.

Where it goes next

- **SCADA ingestion**
Swap assets.json for a live telemetry connection — the scoring logic reads the same shape either way.
- **Trained failure model**
Fit a classifier on historical incident records and keep the identical output contract, so the dashboard never changes.
- **Crew pre-positioning**
Turn the ranked list into a routed dispatch plan across zones ahead of a forecast storm.

What we're proudest of

A complete, working, explainable pipeline — raw sensor and weather data in, a ranked and defensible action plan out — built end to end by someone writing their first real code, with IBM Bob as the development agent for every line of it.

8

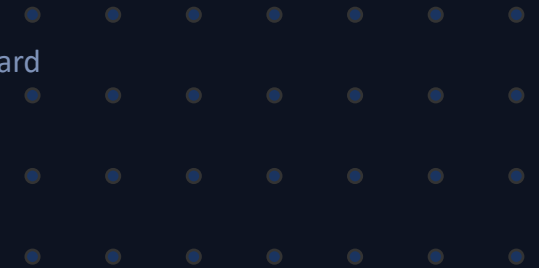
assets scored live

4

Bob-built source files

0

API keys required for the core dashboard



Thank you